

Laporan Tugas Besar 2
Pemanfaatan Algoritma BFS dan DFS dalam
Mekanisme Penelusuran CSS pada Pohon DOM
IF2211 Strategi Algoritma



Kelompok cRUSTacean

13524009 Mikhael Benrael Tampubolon

13524011 Muhammad Iqbal Raihan

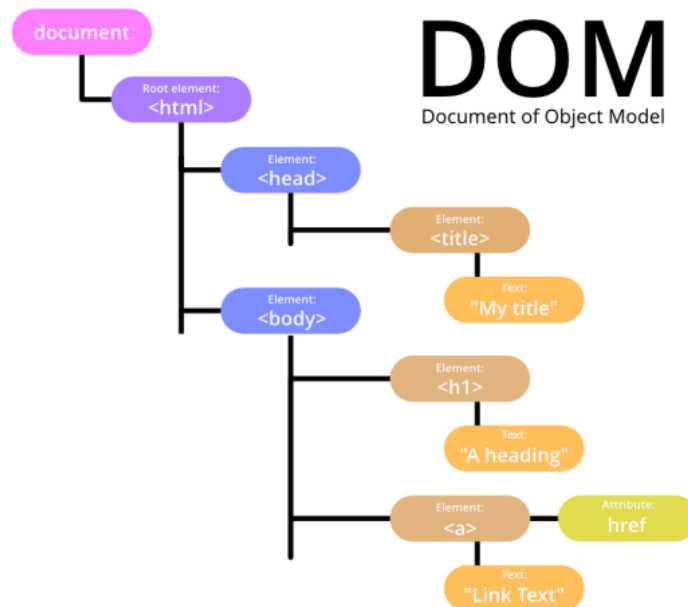
13524099 Muhammad Akmal

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

Daftar Isi

I. Deskripsi Tugas.....	3
II. Landasan Teori.....	7
A. Document Object Model (DOM).....	7
B. HTML Parsing.....	7
1. Tokenisasi.....	7
2. Tree Construction.....	8
C. CSS Selector.....	8
1. Simple Selector.....	8
2. Combinator.....	8
3. CSS Selector Parsing.....	9
D. Algoritma Breadth First Search (BFS).....	9
E. Algoritma Depth First Search (DFS).....	9
F. Multithreading pada Traversal Pohon.....	10
1. Paralelisasi BFS.....	10
2. Paralelisasi DFS.....	10
G. Lowest Common Ancestor (LCA) dengan Binary Lifting.....	10
1. Preprocessing: Euler Tour dan Sparse Table.....	10
2. Binary Lifting.....	11
3. Aplikasi pada DOM.....	11
H. WebAssembly dan Arsitektur Client-Side Rendering (CSR).....	11
A. Algoritma Breadth First Search.....	12
B. Algoritma Depth First Search.....	12
C. Algoritma Lowest Common Ancestor.....	12
III. Implementasi dan Pengujian.....	13
A. Ringkasan Implementasi.....	13
B. Hasil Pengujian.....	13
IV. Kesimpulan dan Saran.....	14
Daftar Pustaka.....	15
Lampiran.....	16
1. Pranala Repository.....	16
2. Tabel Checklist.....	16
3. Pernyataan tidak melakukan kecurangan.....	17

I. Deskripsi Tugas



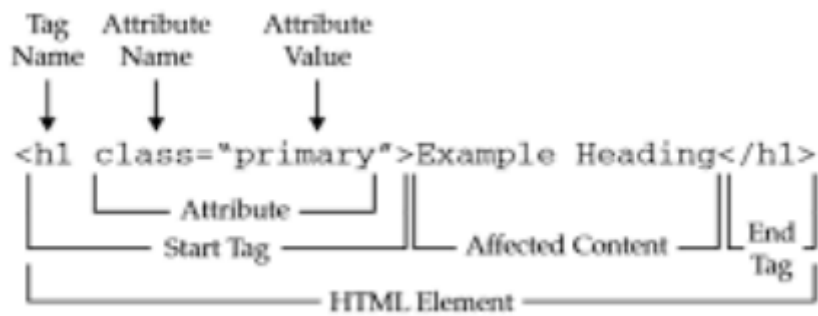
Gambar 1. Struktur Pohon DOM

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram. 1 Struktur sebuah file HTML pada umumnya meliputi elemen root yang berisi dua bagian utama yaitu dan . Bagian berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna. Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempatkan pada elemen-elemen yang berkaitan melalui CSS Selector. Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. Tags

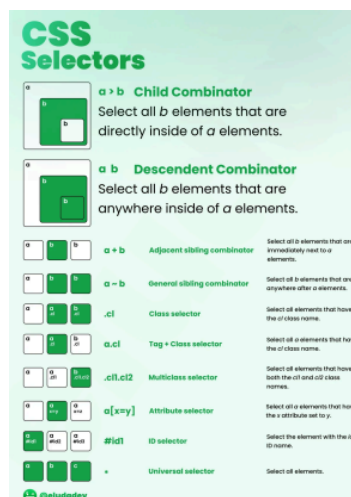


Gambar 2. Struktur Syntax HTML

<https://tutorial.techaltum.com/htmlTags.html>

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web. Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing. Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. CSS Selector



Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

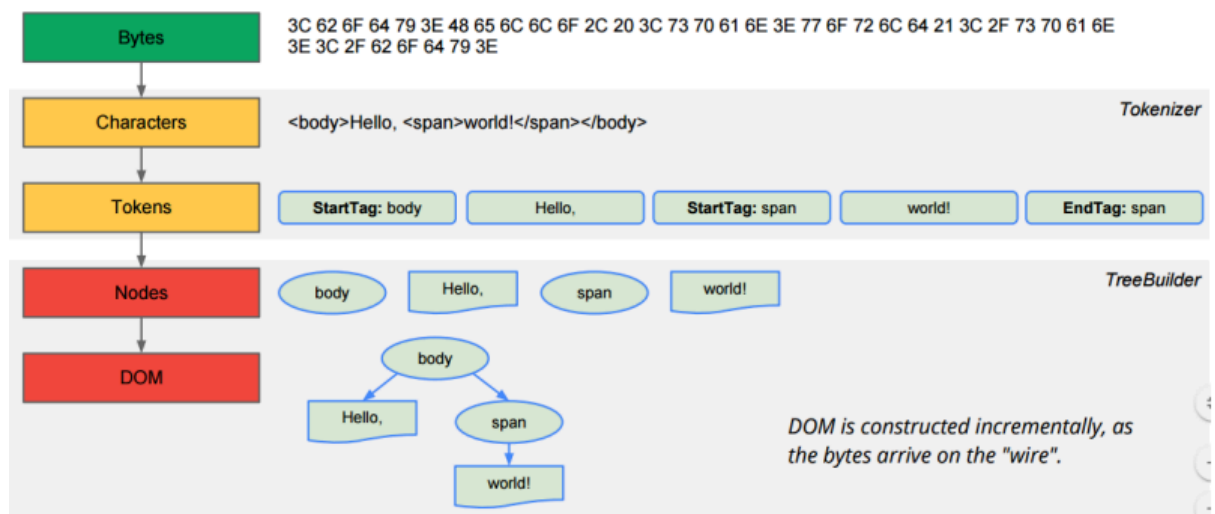
CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- **Tag Selector**
Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen
- **Class Selector** Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.). Contoh: .box
- **ID Selector**
Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#). Contoh: #header
- **Universal Selector**
Memilih semua elemen HTML. Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (" ")
- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)

3. HTML Parsing Gambar



Gambar 4. Proses Parsing Pohon HTML

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen . Dari node

root tersebut, elemen-elemen lain seperti dan akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS)

II. Landasan Teori

A. Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman berbasis standar W3C yang merepresentasikan dokumen HTML atau XML sebagai struktur pohon hierarkis. Setiap komponen dalam dokumen—elemen, atribut, maupun teks—dimodelkan sebagai sebuah node dalam pohon tersebut. Standar ini memungkinkan program untuk mengakses, memodifikasi, menghapus, maupun menambahkan konten dan struktur dokumen secara terprogram.

Dalam konteks dokumen HTML, pohon DOM memiliki satu simpul akar (root) berupa elemen `<html>`, yang bercabang menjadi dua sub-pohon utama: `<head>` dan `<body>`. Elemen `<head>` memuat metadata halaman—seperti judul, referensi stylesheet, dan skrip—sedangkan `<body>` memuat seluruh konten yang dirender kepada pengguna, mulai dari teks dan gambar hingga tombol interaktif.

Hubungan antarsimpul dalam DOM diklasifikasikan sebagai berikut:

- **Parent–Child:** Simpul yang langsung membungkus simpul lain disebut parent, sementara simpul yang berada tepat di dalamnya disebut child.
- **Sibling:** Simpul-simpul yang memiliki parent yang sama berada pada level yang setara dan disebut sibling.
- **Ancestor–Descendant:** Hubungan tidak langsung ke atas (ancestor) dan ke bawah (descendant) dalam lintasan dari root ke sebuah simpul.

Properti pohon DOM yang relevan terhadap tugas ini antara lain kedalaman (depth) setiap simpul, yaitu jarak dari root ke simpul tersebut, serta tinggi (height) pohon yang merepresentasikan kedalaman maksimum di seluruh simpul daun (leaf nodes).

B. HTML Parsing

HTML Parsing adalah proses konversi teks HTML mentah menjadi struktur data pohon DOM yang dapat diolah secara terprogram. Proses ini umumnya terdiri dari dua tahap utama: tokenisasi dan pembangunan pohon (tree construction).

1. Tokenisasi

Tokenizer membaca karakter masukan satu per satu dan menghasilkan token-token bermakna, antara lain:

- **StartTag:** Token pembuka elemen, beserta atribut-atributnya.
- **EndTag:** Token penutup elemen.
- **Character:** Konten teks di antara tag.
- **Comment dan DOCTYPE:** Token khusus yang tidak direpresentasikan sebagai simpul elemen biasa.

2. Tree Construction

Fase ini memanfaatkan tumpukan (stack) elemen terbuka untuk membangun hierarki pohon secara inkremental. Setiap StartTag mendorong simpul baru ke dalam stack; EndTag yang bersesuaian mengeluarkannya. Simpul yang berada dalam stack ketika sebuah simpul baru diproses menjadi parent dari simpul tersebut.

Dalam implementasi cRUSTacean, parsing HTML memanfaatkan library html5ever dan markup5ever yang merupakan implementasi Rust dari spesifikasi HTML5 WHATWG. Library ini menghasilkan token sesuai aturan parsing HTML5, termasuk penanganan markup yang tidak valid. Simpul-simpul yang dihasilkan kemudian disimpan dalam struktur data berbasis vektor di mana setiap simpul diidentifikasi oleh indeksinya, menggantikan penggunaan pointer konvensional yang tidak idiomatis dalam Rust.

C. CSS Selector

CSS Selector adalah sintaks yang digunakan untuk mencocokkan (matching) elemen-elemen tertentu dalam pohon DOM. Dalam konteks pemrosesan CSS, selector dievaluasi dari kanan ke kiri (right-to-left) untuk efisiensi, namun dalam penelusuran pohon DOM berbasis BFS/DFS, selector dievaluasi secara langsung pada setiap simpul yang dikunjungi.

a. Simple Selector

Simple selector adalah unit terkecil yang mencocokkan satu elemen berdasarkan satu kriteria:

- Tag Selector: Mencocokkan elemen berdasarkan nama tag-nya. Contoh: div mencocokkan semua elemen
- Class Selector (.): Mencocokkan elemen yang memiliki nilai atribut class tertentu. Contoh: .container mencocokkan semua elemen dengan class "container". Satu elemen dapat memiliki lebih dari satu class.
- ID Selector (#): Mencocokkan tepat satu elemen berdasarkan nilai atribut id-nya. Contoh: #navbar. ID bersifat unik dalam satu dokumen.
- Universal Selector (*): Mencocokkan semua elemen tanpa pengecualian.

b. Combinator

Combinator menghubungkan dua simple selector dan mendefinisikan relasi posisional di antara elemen yang dicocokkan:

- Descendant Combinator (spasi): Mencocokkan elemen B yang merupakan keturunan (di kedalaman berapa pun) dari elemen A. Contoh: div p.
- Child Combinator (>): Mencocokkan elemen B yang merupakan child langsung dari elemen A. Contoh: ul > li.
- Adjacent Sibling Combinator (+): Mencocokkan elemen B yang merupakan sibling langsung berikutnya setelah elemen A (keduanya memiliki parent yang sama). Contoh: h1 + p.
- General Sibling Combinator (~): Mencocokkan elemen B yang merupakan sibling dari elemen A dan muncul setelahnya, tidak harus langsung. Contoh: h1 ~ p.

c. CSS Selector Parsing

Dalam implementasi cRUSTacean, selector diurai oleh package css-selector menjadi representasi terstruktur yang terdiri dari urutan simple selector dan combinator. Proses parsing menghasilkan pohon selector yang kemudian dievaluasi selama penelusuran pohon DOM. Tokenisasi selector membedakan karakter ., #, *, >, +, dan ~ dari nama tag biasa untuk menentukan jenis selector yang tepat.

D. Algoritma Breadth First Search (BFS)

Breadth First Search (BFS) adalah algoritma penelusuran graf yang mengeksplorasi semua simpul pada level kedalaman yang sama sebelum bergerak ke level berikutnya. BFS menggunakan struktur data antrian (queue) yang bersifat FIFO (First In, First Out) sebagai mekanisme utamanya. BFS bekerja dengan langkah-langkah berikut:

1. Masukkan simpul akar (root) ke dalam antrian dan tandai sebagai dikunjungi.
2. Selama antrian tidak kosong, ambil simpul terdepan dari antrian.
3. Proses simpul tersebut (dalam konteks tugas ini: evaluasi apakah simpul cocok dengan selector).
4. Masukkan semua anak (children) dari simpul yang diproses ke bagian belakang antrian.
5. Ulangi langkah 2–4.

Kompleksitas waktu BFS adalah $O(V + E)$ di mana V adalah jumlah simpul dan E adalah jumlah sisi. Pada pohon DOM, setiap simpul memiliki tepat satu parent (kecuali root), sehingga $E = V - 1$ dan kompleksitas efektifnya adalah $O(V)$. Kompleksitas ruang adalah $O(w)$ di mana w adalah lebar maksimum pohon (jumlah simpul pada level terlebar), karena antrian perlu menyimpan seluruh level tersebut secara bersamaan. BFS menelusuri pohon DOM secara level-by-level, sehingga elemen-elemen yang ditemukan lebih awal cenderung berada lebih dekat dengan root. Ini membuat BFS lebih sesuai ketika elemen target diperkirakan berada di kedalaman yang dangkal, atau ketika urutan hasil berdasarkan kedekatan dengan root diprioritaskan.

E. Algoritma Depth First Search (DFS)

Depth First Search (DFS) adalah algoritma penelusuran yang mengeksplorasi setiap cabang pohon sedalam mungkin sebelum melakukan backtracking ke simpul sebelumnya. DFS menggunakan tumpukan (stack)—baik secara eksplisit maupun melalui rekursi—sebagai mekanisme utamanya. DFS bekerja dengan langkah-langkah berikut:

1. Kunjungi simpul awal (root).
2. Tandai simpul tersebut sebagai dikunjungi dan proses.
3. Pilih salah satu anak yang belum dikunjungi, kemudian rekursif lakukan DFS dari anak tersebut.
4. Jika semua anak telah dikunjungi, lakukan backtracking ke simpul parent.
5. Ulangi hingga seluruh pohon terkunjungi.

Dalam pohon DOM, DFS umumnya dilakukan secara pre-order (kunjungi simpul sebelum anak-anaknya), yang menghasilkan urutan penelusuran yang merepresentasikan urutan elemen HTML dari atas ke bawah dalam kode sumber.

Kompleksitas waktu DFS juga $O(V)$ pada pohon. Kompleksitas ruang DFS adalah $O(h)$ di mana h adalah tinggi pohon, karena tumpukan rekursi atau stack eksplisit hanya perlu menyimpan simpul-simpul sepanjang satu lintasan dari root ke simpul saat ini. DFS lebih efisien secara memori dibandingkan BFS pada pohon yang sangat lebar namun tidak terlalu dalam. DFS cocok ketika elemen target diperkirakan berada di cabang yang dalam, atau ketika keseluruhan sub-pohon suatu elemen perlu dieksplorasi secara penuh.

F. Multithreading pada Traversal Pohon

Multithreading adalah teknik pemrograman konkuren yang memungkinkan beberapa thread berjalan secara bersamaan dalam satu proses, memanfaatkan kapabilitas prosesor multi-inti. Penerapan multithreading pada traversal pohon DOM bertujuan untuk mempercepat pencarian dengan memproses beberapa subtree secara paralel.

a. Paralelisasi BFS

Pada BFS, setiap level pohon dapat diproses secara paralel karena simpul-simpul dalam satu level bersifat independen satu sama lain. Pendekatan ini dikenal sebagai level-synchronous BFS. Thread-thread yang berbeda dapat mengambil simpul dari antrian bersama secara konkuren, dengan sinkronisasi yang diperlukan untuk menghindari kondisi balapan (race condition).

b. Paralelisasi DFS

Pada DFS, subtree dari setiap anak simpul akar dapat dieksplorasi oleh thread yang berbeda secara independen. Karena setiap subtree tidak memiliki tumpang tindih simpul, sinkronisasi antar-thread hanya diperlukan saat menggabungkan hasil akhir, bukan selama traversal berlangsung. Pendekatan ini efisien untuk pohon yang memiliki banyak subtree besar.

G. Lowest Common Ancestor (LCA) dengan Binary Lifting

Lowest Common Ancestor (LCA) dari dua simpul u dan v dalam suatu pohon adalah simpul dengan kedalaman terbesar yang merupakan leluhur (ancestor) dari keduanya. Dalam konteks pohon DOM, LCA digunakan untuk menemukan elemen HTML terdekat yang menjadi container bersama dari dua elemen tertentu, misalnya menentukan elemen pembungkus bersama yang paling spesifik.

1. Preprocessing: Euler Tour dan Sparse Table

Pendekatan klasik LCA memanfaatkan Euler Tour (atau DFS order) untuk merekam urutan kunjungan simpul beserta kedalamannya. Masalah LCA kemudian direduksi menjadi Range Minimum Query (RMQ) pada array kedalaman hasil Euler Tour.

2. Binary Lifting

Binary Lifting adalah teknik preprocessing yang membangun tabel ancestor dua dimensi, di mana $up[v][k]$ menyimpan leluhur dari simpul v yang berada 2^k level di atasnya. Preprocessing dilakukan dalam waktu $O(V \log V)$ dan memerlukan ruang $O(V \log V)$. Algoritma LCA dengan Binary Lifting bekerja sebagai berikut:

1. Lakukan DFS untuk mencatat kedalaman setiap simpul dan parent langsungnya ($up[v][0]$).
2. Isi tabel Binary Lifting dengan relasi: $up[v][k] = up[up[v][k-1]][k-1]$.
3. Untuk query LCA(u, v): samakan kedalaman keduanya dengan mengangkat simpul yang lebih dalam, kemudian angkat keduanya bersama hingga bertemu pada ancestor yang sama.

3. Aplikasi pada DOM

Dalam pohon DOM, LCA memiliki aplikasi praktis seperti menentukan elemen yang perlu distyle secara bersama ketika dua elemen dipilih oleh selector yang berbeda, atau menganalisis hubungan struktural antara dua elemen yang ditemukan selama penelusuran.

H. WebAssembly dan Arsitektur Client-Side Rendering (CSR)

WebAssembly (WASM) adalah format bytecode biner yang dapat dieksekusi oleh browser modern dengan performa mendekati kode native. WASM dirancang sebagai target kompilasi untuk bahasa-bahasa seperti C, C++, dan Rust, memungkinkan kode yang ditulis dalam bahasa tersebut berjalan langsung di browser. Keuntungan arsitektur CSR pada konteks ini antara lain:

- Latensi rendah: Tidak diperlukan round-trip ke server untuk setiap operasi traversal.
- Skalabilitas: Server tidak dibebani oleh komputasi intensif; beban didistribusikan ke browser pengguna.
- Keamanan: Data HTML yang di-scrape diproses sepenuhnya di sisi klien tanpa melewati server (kecuali saat fetching awal).

Backend berperan sebagai proxy sederhana yang meneruskan permintaan HTTP ke URL target dan mengembalikan konten HTML-nya ke frontend. Hal ini diperlukan karena browser membatasi permintaan lintas domain (CORS) secara langsung.

III. Aplikasi BFS dan DFS

Kedua algoritma ini berfungsi sebagai mesin pencarian utama untuk menelusuri setiap node pada pohon DOM dan mencocokkannya dengan CSS Selector. BFS lebih efisien jika elemen yang dicari berada dekat dengan root, sedangkan DFS menelusuri secara mendalam dari atas ke bawah seperti urutan struktur HTML. BFS bekerja secara level per level, sehingga semua node pada satu tingkat diperiksa terlebih dahulu sebelum turun ke tingkat berikutnya. Sementara itu, DFS akan langsung menelusuri satu cabang hingga ke node terdalam sebelum berpindah ke cabang lain.

A. Algoritma Breadth First Search

Implementasi BFS (algorithm/src/bfs.rs) memanfaatkan antrean dua ujung atau VecDeque yang beroperasi dengan prinsip *First-In-First-Out* (FIFO). Pendekatan ini memastikan bahwa simpul pada level kedalaman yang lebih dangkal akan dieksekusi dan dievaluasi kecocokannya terhadap CSS Selector terlebih dahulu sebelum memproses simpul di level yang lebih dalam.

Selain itu, algoritma ini dirancang dengan mekanisme penghentian dini (*early exit*). Jika pengguna memberikan batasan jumlah hasil (*top_n*), penelusuran akan langsung dihentikan setelah target jumlah terpenuhi untuk menghemat waktu komputasi.

```
pub fn bfs(tree: &DomTree, sel: &ComplexSelector, top_n: Option<usize>) -> AlgorithmResult {  
    // ... [Inisialisasi TreeIndex, array steps, array matched, dsb.] ...  
  
    let mut q = VecDeque::new();  
  
    // 1. Inisialisasi: Masukkan simpul akar (root) ke dalam antrean FIFO.  
    // Nilai `None` menandakan bahwa root tidak memiliki simpul parent asal.  
    q.push_back((root, None));  
  
    // 2. Loop akan terus memproses simpul selama antrean tidak kosong.  
    while let Some((node, from)) = q.pop_front() {  
        // 3. Evaluasi apakah simpul saat ini memenuhi aturan CSS Selector.  
        let is_match = matches(node, &ti, sel);  
  
        // 4. Catat setiap simpul yang dikunjungi ke traversal log.  
        steps.push(TraversalStep { step: steps.len(), node_index: node, from_index: from,  
            is_match });  
  
        // 5. Jika simpul cocok dengan selector, simpan indeksinya.
```

```

        if is_match {
            matched.push(node);
        }
    }

    // 6. Hentikan loop jika hasil sudah mencapai batasan top_n.
    if Some(matched.len()) == top_n { break; }
}

// 7. Masukkan seluruh anak dari simpul saat ini ke bagian belakang antrean.
for &c in &tree.nodes[node].children {
    q.push_back((c, Some(node)));
}

}

// ... [Return AlgorithmResult] ...
}

```

B. Algoritma Depth First Search

Algoritma DFS (algorithm/src/dfs.rs) dibangun menggunakan pendekatan iteratif dengan tumpukan (Stack) berbasis *Last-In-First-Out* (LIFO) untuk mencegah *stack overflow* pada pohon yang sangat dalam.

Keunikan logika DFS di sini terletak pada manipulasi urutan anak yang dimasukkan ke dalam tumpukan. Agar aplikasi meniru penelusuran dokumen HTML secara natural dari kiri ke kanan (*pre-order*), daftar simpul anak harus dibalik (*di-reverse*) sebelum dimasukkan, sehingga anak paling kiri berada di puncak tumpukan dan dieksekusi pertama kali.

```

pub fn dfs(tree: &DomTree, sel: &ComplexSelector, top_n: Option<usize>) -> AlgorithmResult {
    // ... [Inisialisasi struktur] ...

    // 1. Inisialisasi: Menggunakan Vector bawaan Rust sebagai Stack (LIFO).
    let mut stack = vec![(root, None)];

    // 2. Ekstraksi selalu mengambil simpul yang paling terakhir dimasukkan ke stack.
    while let Some((node, from)) = stack.pop() {
        let is_match = matches(node, &ti, sel);

        // 3. Pencatatan log dan Early-Exit persis sama seperti logika pada BFS.
    }
}

```

```

        steps.push(TraversalStep { /* ... */ });
    if is_match {
        matched.push(node);
        if Some(matched.len()) == top_n { break; }
    }

// 4. Penanganan Anak (Children Handling)

// Kita mengiterasi daftar anak secara terbalik (.iter().rev()). Karena stack bersifat LIFO, anak
// yang terakhir di-push akan menjadi yang pertama di-pop.

    for &c in tree.nodes[node].children.iter().rev() {
        stack.push((c, Some(node)));
    }
}

// ... [Return AlgorithmResult] ...
}

```

C. Algoritma Lowest Common Ancestor

Implementasi LCA (algorithm/src/lca.rs) menggunakan teknik komputasi dinamis yang disebut *Binary Lifting*. Teknik ini memecah pencarian melalui pra pemrosesan tabel logaritma up, yang menyimpan leluhur ke- 2^k untuk setiap simpul.

Saat melihat LCA dari dua simpul, algoritma ini tidak berjalan naik satu tingkat demi satu tingkat secara linier. Algoritma terlebih dahulu menyamakan tingkat kedalaman kedua simpul, lalu secara serentak melompat tinggi memangkas jarak eksponensial ke atas secara simultan, sehingga kompleksitas pencariannya sangat efisien.

D. Implementasi Multithreading pada BFS dan DFS

Pemenuhan spesifikasi bonus multithreading diimplementasikan menggunakan *crate* rayon untuk mendistribusikan beban kerja penelusuran secara paralel ke beberapa *thread*. Pada algoritma BFS, paralelisasi dilakukan dengan pendekatan *level-synchronous*, di mana seluruh simpul yang berada pada tingkat kedalaman yang sama (satu antrean) dievaluasi secara serentak oleh *thread pool* sebelum berpindah ke tingkat selanjutnya. Sementara itu, algoritma DFS dirombak menjadi rekursif paralel menggunakan pendekatan *divide-and-conquer*; anak-anak dari sebuah simpul dipecah menjadi dua bagian yang masing-masing dieksekusi secara konkuren melalui `rayon::join`.

Karena penelusuran ini melibatkan banyak *thread* yang mencatat jejak kunjungan dan hasil pencocokan secara serentak, potensi terjadinya tabrakan data (*data race*) sangat tinggi. Untuk menangani hal ini, diimplementasikan struktur Recorder yang terisolasi. Struktur ini mengamankan area memori (*shared state*) menggunakan pengunci Mutex untuk data vektor dan operasi variabel atomik (AtomicUsize, AtomicBool) untuk perhitungan konstan, sehingga menjamin status *early-exit* dan manipulasi log aman terhadap kondisi *thread-safe*.

IV. Implementasi dan Pengujian

A. Ringkasan Implementasi

Sistem dibagi menjadi beberapa *crate* independen. Setiap *crate* memiliki peran yang spesifik dalam proses *scraping*, manipulasi data, dan representasi visual.

a. algorithm

Evaluasi CSS *selector* pada sebuah *node* dilakukan menggunakan *right-to-left matching*. Algoritma ini akan langsung menolak *node* yang tidak cocok dengan kriteria utama di posisi paling kanan untuk menghemat waktu komputasi. Fungsi **matches** mengevaluasi target yang paling spesifik terlebih dahulu. Jika target tersebut cocok, proses dilanjutkan dengan *men-traverse* rantai *parent* atau melompat antar *sibling* menggunakan fungsi **walk_left**. Proses ini menggunakan data statis pada **TreeIndex** untuk mengeksekusi logika *combinator* seperti *descendant*, *child*, *adjacent sibling*, dan *general sibling*.

```
// crates/algorithm/src/matcher.rs
pub fn matches(idx: usize, ti: &TreeIndex, sel:
&ComplexSelector) -> bool {
    let n_seq = 1 + sel.rest.len();
    let last = n_seq - 1;
    if !matches_sequence(idx, ti, seq_at(sel, last)) { return
false; }

    if last == 0 { return true; }

    walk_left(last - 1, idx, idx, ti, sel)
}
```

Untuk mempercepat proses pengecekan relasi *parent-child* pada logika *combinator*, struktur **TreeIndex** menggunakan *Lowest Common Ancestor* (LCA) dengan teknik *binary lifting*. Sebuah *logarithmic matrix* dibuat pada awal komputasi. Melalui struktur ini, jarak antar *node* dapat dihitung dengan cepat lewat operasi *bitwise*, sehingga program terhindar dari *linear scan* yang memperlambat *traversal*.

```
// crates/algorithm/src/lca.rs
pub fn lca(&self, mut a: usize, mut b: usize) -> usize {
    let mut diff = self.depth[a] - self.depth[b];
    let mut bit = 0usize;
    while diff > 0 {
        if diff & 1 != 0 {
            a = self.up[a][bit];
        }
    }
}
```



```

        diff >>= 1;
        bit += 1;
    }
    // mengangkat kedua node bersamaan jika belum mencapai
    ancestor
}

```

b. backend

Crate ini menggunakan *framework* Axum untuk menyediakan REST API *endpoints*. Pemrosesan *web scraping* diatur oleh *library* HTTP client. Pengambilan data HTML dilindungi menggunakan validasi tautan URL dan limitasi memori. Respon dari target web yang diambil kemudian akan di-*parse* menggunakan *crate* internal dan diserialisasi kembali ke format JSON.

```

// crates/backend/src/scrape.rs
pub async fn read_response_body_limited(mut response:
request::Response, max_body_bytes: usize) -> Result<String,
ApiError> {
    let mut body = Vec::new();
    while let Some(chunk) =
response.chunk().await.map_err(|error|
ApiError::Upstream(format!("error: {error}")))? {
        if body.len().saturating_add(chunk.len()) >
max_body_bytes {
            return Err(ApiError::Upstream(format!("exceeds
limit")));
        }
        body.extend_from_slice(&chunk);
    }
    // konversi byte array menjadi string UTF-8
}

```

c. css-selector

Crate ini digunakan untuk mengekstraksi *raw string* dari pengguna menjadi struktur **ComplexSelector**. *Parser* ini diimplementasikan melalui *Deterministic Finite Automaton* (DFA). Setiap karakter dievaluasi menggunakan status bacaan tertentu, sehingga ekstraksi nama *class*, *id*, maupun elemen *tag* dilakukan secara sistematis tanpa harus menggunakan *library* regex.

```

// crates/css-selector/src/lib.rs
match c {

```

```

    '.' => {
        // flush data buffer sebelumnya ke dalam selector
sequence
        flush!();
        chars.next();
        match chars.peek() {
            None | Some(' ') | Some('>') | Some('+') |
Some('~') | Some('.') | Some('#') => {
                return
Err(ParseError::EmptySelectorName('.'));
            }
            _ => {}
        }
        state = State::ReadingClass;
    }
    // pemrosesan state lain
}

```

d. frontend

Aplikasi client diimplementasikan melalui *framework* Leptos yang dikompilasi secara menyeluruh menjadi WebAssembly (WASM). Antarmuka *web* dibagi menjadi tiga macam *view stage*. Semua bentuk komunikasi instruksi dengan backend REST API dijalankan secara *asynchronous* melalui *crate* **gloo_net**.

Interaktivitas visual pada komponen halaman web ini diotomatisasi dengan reactive signals. Nilai parameter yang terbaru, seperti status *load spinner* saat *request* terkirim, di-*render* ulang menuju komponen UI di *browser* pengguna.

```

// crates/frontend/src/api.rs
pub async fn scrape_tree(url: &str) ->
Result<ScrapeTreeResponse, String> {
    let encoded_url = urlencoding::encode(url);
    let endpoint =
format!("/scrape?url={}&include_html=false", encoded_url);

    let response = Request::get(&endpoint)
        .send()
        .await
        .map_err(|e| format!("Network error: {e}"))?;

    read_json::<ScrapeTreeResponse>(response).await
}

```

e. html-parser

Proses parsing HTML memanfaatkan **tokenizer** dari crate **html5ever** untuk memproses *raw string* HTML menjadi *semantic tokens*. Penyusunan DOM tree dilakukan secara *custom* melalui **struct Builder**. *Struct* ini menggunakan sebuah *explicit open-element stack*. Pendekatan ini dibangun tanpa memakai *tree builder* bawaan **html5ever**.

```
// crates/html-parser/src/lib.rs
pub fn parse_with_options(html: &str, opts: &ParseOptions) ->
Result<DomTree, ParseError> {
    let sink = Html5Sink::new(Builder::new(opts.clone()));
    let mut tokenizer = Tokenizer::new(sink,
TokenizerOpts::default());
    // iterasi antrean token ke dalam sink
    builder.finish()
}
```

Penggunaan *explicit open-element stack* ini juga mempermudah *error recovery* secara *on-the-fly*. Sebagai contoh, saat elemen **<p>** baru dibaca, *stack* dapat dievaluasi secara langsung. Jika masih ada *tag <p>* yang terbuka di posisi teratas *stack*, *tag* tersebut akan di-*pop* terlebih dahulu untuk mencegah elemen bertumpuk menjadi *nested paragraph* yang merusak hierarki DOM *tree*.

```
// crates/html-parser/src/builder.rs
fn close_open_p_for_new_p(&mut self) {
    while self.open.len() > 1 {
        let idx = *self.open.last().expect("non-empty");
        let is_p = matches!(
            &self.tree.nodes[idx].data,
            NodeData::Element { tag_name, .. } if
tag_name.eq_ignore_ascii_case("p")
        );
        if !is_p { break; }
        self.open.pop();
    }
}
```

f. shared

Crate ini digunakan untuk mendefinisikan semua struktur data fundamental yang dipakai oleh aplikasi, seperti **DomTree** dan **NodeData**. Data disusun menggunakan skema indeks datar berbasis array (**Vec<Node>**) agar lebih *cache-friendly*. Pendekatan ini dilakukan tanpa memakai memori referensi *pointer*

sehingga data *tree* dapat diserialisasi menjadi format JSON dengan mudah untuk komunikasi antara *server* dan *client*.

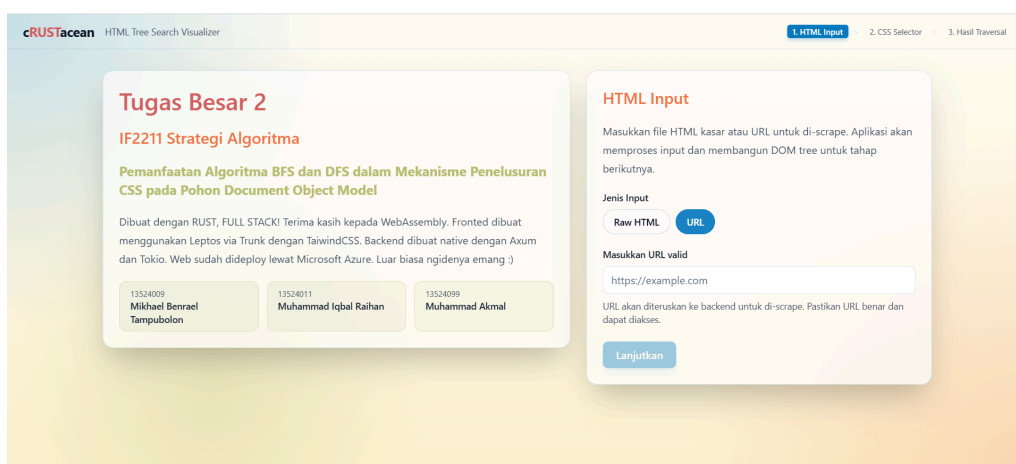
```
// crates/shared/src/lib.rs
pub fn add_node(&mut self, data: NodeData, parent:
Option<usize>) -> Result<usize, String> {
    // kalkulasi depth berdasarkan parent node
    let index = self.nodes.len();
    if self.nodes.is_empty() { self.root = Some(index); }

    self.nodes.push(Node {
        index,
        parent,
        children: Vec::new(),
        data,
        depth,
    });
    // menyambungkan node ke parent index secara otomatis
}
```

B. Hasil Pengujian

Dilakukan pengujian dengan beberapa skenario sebagai berikut.

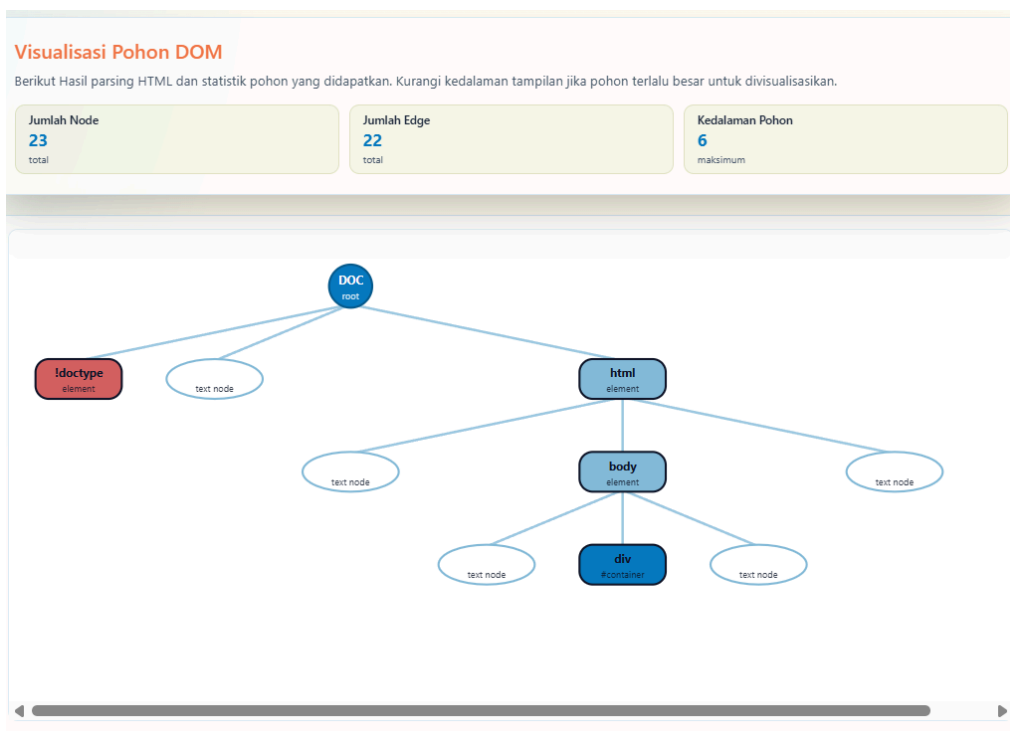
a. Halaman utama dari *website*

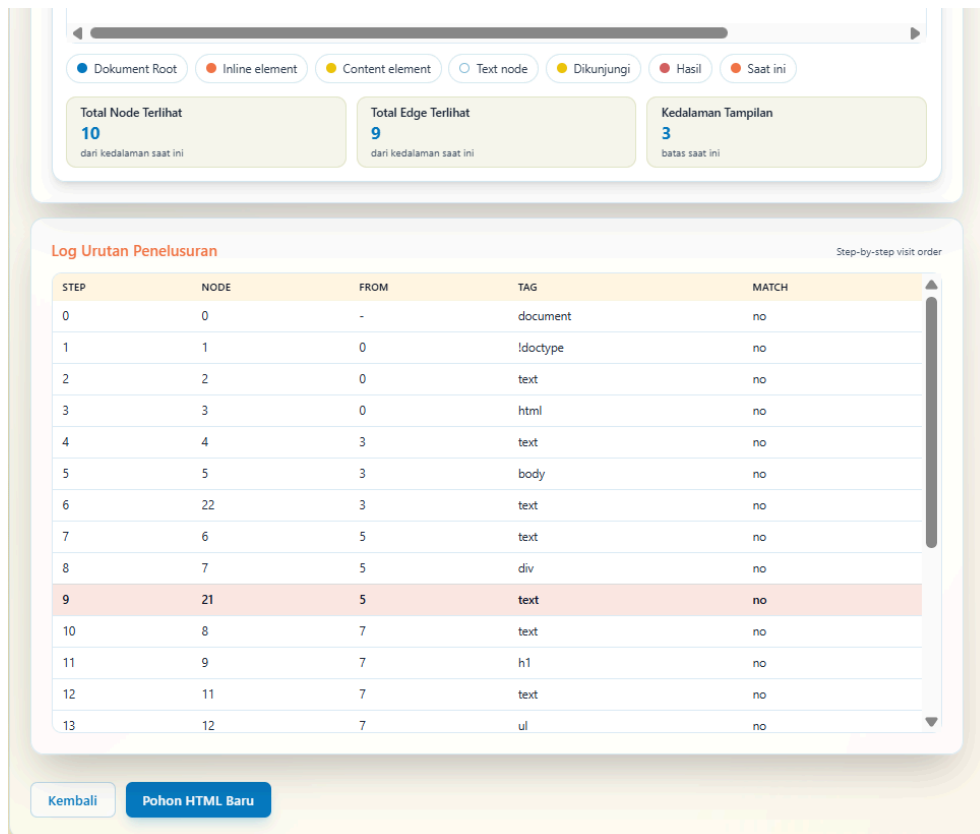
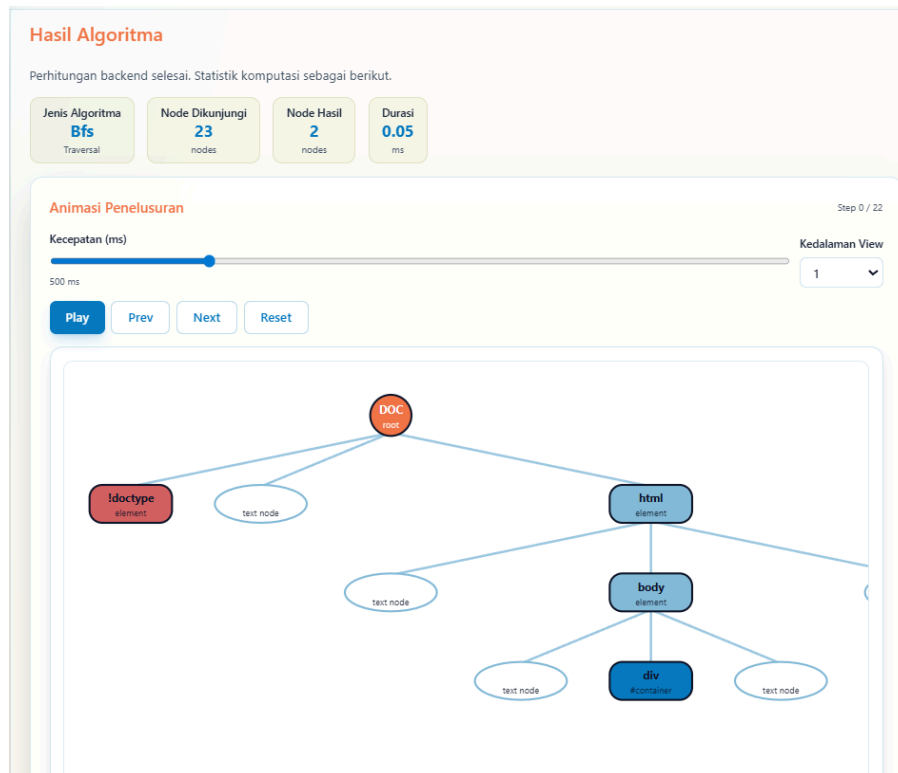


b. Pengujian menggunakan *raw* HTML dengan BFS

```
<!DOCTYPE html>
<html>
<body>
<div id="container">
<h1 class="title">Judul</h1>
<ul class="list">
<li>Item A</li>
<li>Item B</li>
</ul>
</div>
</body>
</html>
```

Hasil:



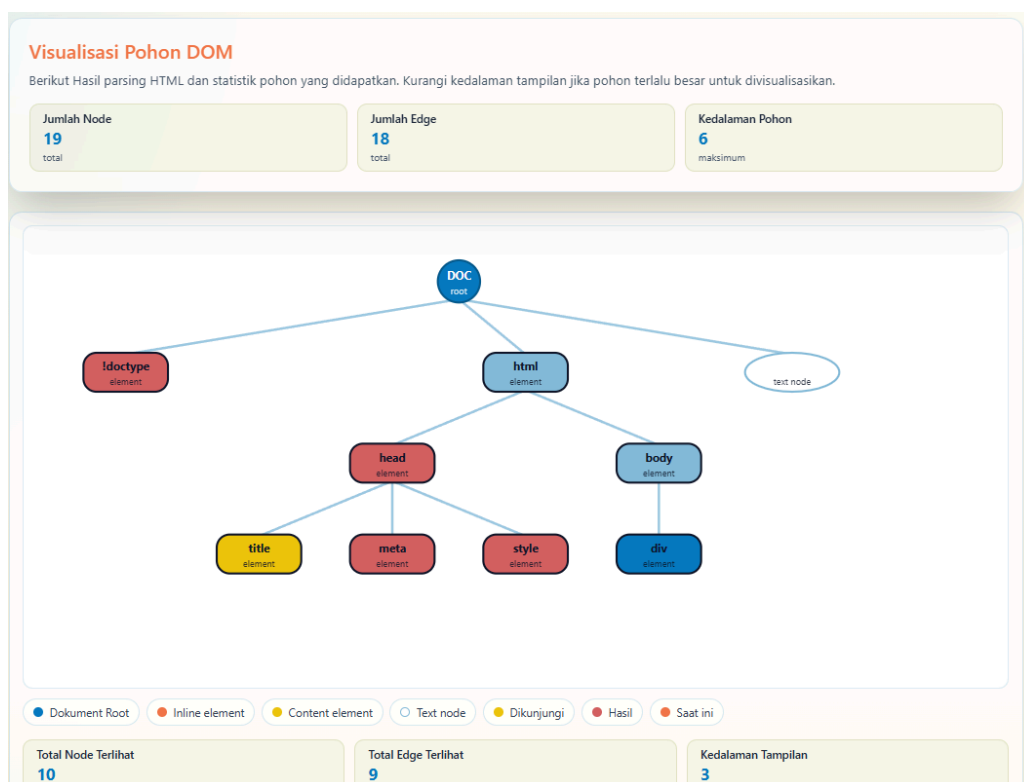


Log Urutan Penelusuran Step-by-step visit order

STEP	NODE	FROM	TAG	MATCH
9	21	5	text	no
10	8	7	text	no
11	9	7	h1	no
12	11	7	text	no
13	12	7	ul	no
14	20	7	text	no
15	10	9	text	no
16	13	12	text	no
17	14	12	li	yes
18	16	12	text	no
19	17	12	li	yes
20	19	12	text	no
21	15	14	text	no
22	18	17	text	no

c. Pengujian menggunakan *website* statik dengan DFS

<https://example.com/>



Hasil Algoritma

Perhitungan backend selesai. Statistik komputasi sebagai berikut.

Jenis Algoritma
Dfs
Traversal

Node Dikunjungi
19
nodes

Node Hasil
0
nodes

Durasi
0.74
ms

Animasi Penelusuran

Step 0 / 18

Kecepatan (ms)

500 ms

Kedalaman View

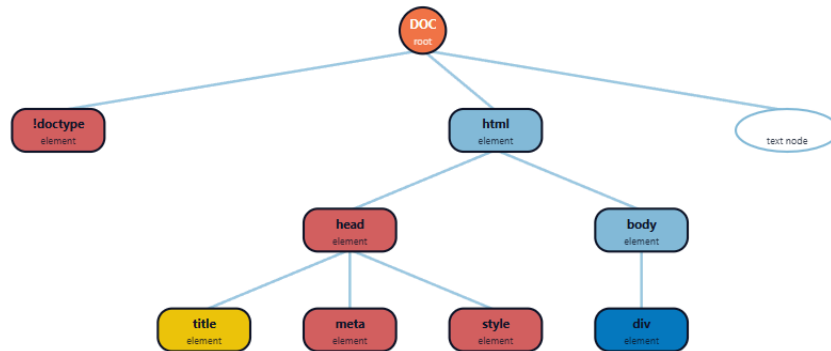
1

Play

Prev

Next

Reset



● Dokument Root

● Inline element

● Content element

○ Text node

● Dikunjungi

● Hasil

● Saat ini

Total Node Terlihat

10

dari kedalaman saat ini

Total Edge Terlihat

9

dari kedalaman saat ini

Kedalaman Tampilan

3

batas saat ini

Log Urutan Penelusuran

Step-by-step visit order

STEP	NODE	FROM	TAG	MATCH
0	0	-	document	no
1	1	0	!doctype	no
2	2	0	html	no
3	3	2	head	no
4	4	3	title	no
5	9	2	body	no
6	5	4	text	no
7	10	9	div	no
8	6	3	meta	no
9	11	10	h1	no
10	13	10	p	no
11	12	11	text	no
12	7	3	style	no
13	14	13	text	no

d. Pengujian dengan URL menggunakan LCA

<https://www.youtube.com/watch?v=dQw4w9WgXcQ>

Visualisasi Pohon DOM

Berikut Hasil parsing HTML dan statistik pohon yang didapatkan. Kurangi kedalaman tampilan jika pohon terlalu besar untuk divisualisasikan.

Jumlah Node
398
total

Jumlah Edge
397
total

Kedalaman Pohon
10
maksimum

DOC
root

file
element

meta
element

meta
element

meta
element

link
element

link
element

link
element

link
element

Dokument Root

Inline element

Content element

Text node

Dikunjungi

Hasil

Saat ini

Total Node Terlihat
162

Total Edge Terlihat
161

Kedalaman Tampilan
2

Dokument Root

Inline element

Content element

Text node

Dikunjungi

Hasil

Saat ini

Total Node Terlihat
162
dari kedalaman saat ini

Total Edge Terlihat
161
dari kedalaman saat ini

Kedalaman Tampilan
3
batas saat ini

CSS Selector

Masukkan selector, jenis algoritma, and batasan hasil sebelum melanjutkan.

Mode Run

Traversal

LCA

Kedalaman Tampilan Pohon
1

CSS Selector
div#container li

Notasi yang didukung: tag, .class, #id, descendant, child (>), adjacent (+), sibling (~).

Jenis Algoritma

BFS

DFS

Paralellisme

Single Thread

Multi Thread

Batasan Hasil

Semua

Top N

Kembali

Jalankan Algoritma Traversal

Hasil Algoritma

Didapatkan Lowest Common Ancestor (LCA) antara Node A dan Node B sebagai berikut.

Node A
2
Input (kuning)

Node B
4
Input (kuning)

LCA
2
leluhur terdekat (yang jingga ya)

Lowest Common Ancestor ditemukan

Visualisasi Pohon DOM Kedalaman View 1

doctype element

script element

meta element

meta element

script element

script element

script element

Legend: ● Dokument Root ● Inline element ● Content element ● Text node ● Dikunjungi ● Hasil ● Saat ini

V. Kesimpulan dan Saran

A. Kesimpulan

Aplikasi penelusuran DOM tree ini berhasil dikembangkan dengan arsitektur yang sangat modular melalui pemisahan menjadi beberapa modul independen. Pendekatan ini secara efektif memisahkan peran masing-masing komponen. Pemisahan peran tersebut mencakup tokenisasi HTML, parser selektor CSS, pengelolaan HTTP, serta antarmuka depan berbasis WebAssembly. Akibatnya, Seluruh modul beroperasi secara stabil dan terisolasi dari potensi hambatan sistem.

Penerapan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) telah berhasil diintegrasikan untuk mengevaluasi data pohon HTML dengan akurat. Penelusuran ini juga disempurnakan oleh kapabilitas multithreading parallel berbasis Rayon yang terbukti sangat mempercepat kinerja traversal pada website berukuran besar tanpa memunculkan race condition. Selain itu, metode right-to-left matching yang dipercepat oleh indexing sehingga Lowest Common Ancestor (LCA) berhasil memangkas waktu komputasi saat program memecahkan logika kombinator CSS. Berdasarkan hasil pengujian komprehensif, disimpulkan bahwa program ini telah mematuhi seluruh spesifikasi wajib dan tambahan yang ditugaskan.

B. Saran

Meskipun program telah berjalan dengan sangat efisien, beberapa pengembangan lanjutan masih dapat dipertimbangkan untuk agenda di masa depan. Algoritma parsing pada selektor CSS dapat diperluas fungsinya agar mampu mengenali metode selektor modern yang lebih kompleks. Penambahan tersebut dapat berupa dukungan terhadap pseudo-classes maupun attribute selektor.

Selain itu, modul error recovery pada HTML parser dapat ditingkatkan lagi agar program lebih toleran dalam memperbaiki struktur sintaks yang buruk pada dokumen HTML dari berbagai situs web lawas yang tidak terstandarisasi. Terakhir, mekanisme lalu lintas data payload antara backend dan frontend dapat dioptimalkan lebih lanjut. Optimalisasi ini dapat dilakukan dengan menerapkan mekanisme byte streaming atau data compression untuk meminimalisasi waktu jeda saat aplikasi diinstruksikan melakukan web scraping pada tautan dengan ukuran file yang ekstrem.

Daftar Pustaka

- [1] World Wide Web Consortium, "Document Object Model (DOM)," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Document_Object_Model.
- [2] MDN Web Docs Contributors, "CSS Selectors," MDN Web Docs, Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors>.
- [3] WHATWG, "HTML Living Standard — Parsing HTML documents," WHATWG Living Standard. [Online]. Available: <https://html.spec.whatwg.org/multipage/parsing.html>.
- [4] R. Munir, "BFS dan DFS (Bagian 1)," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2026)-Bagian1.pdf).
- [5] R. Munir, "BFS dan DFS (Bagian 2)," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-(2026)-Bagian2.pdf)
- [6] The Rust and WebAssembly Working Group, "Rust and WebAssembly," rustwasm GitHub, 2024. [Online]. Available: <https://rustwasm.github.io/book/>.
- [7] WebAssembly Community Group, "WebAssembly," webassembly.org, 2024. [Online]. Available: <https://webassembly.org/>.
- [8] The Rust Programming Language Team, "WebAssembly — Rust Programming Language," rust-lang.org, 2024. [Online]. Available: <https://www.rust-lang.org/what/wasm>.

Lampiran

1. Pranala Penting

- Tautan repository: https://github.com/m-akmal/Tubes2_cRUSTacean.git
- Tautan web deploy: https://s.hmif.dev/Tubes2_cRUSTacean

2. Tabel Checklist

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	

3. Pernyataan tidak melakukan kecurangan

Tugas ini disusun tanpa sepenuhnya menggunakan bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, appearing to read 'mikh' with a stylized flourish at the end.

Mikhael Benrael Tampubolon

A handwritten signature in black ink, consisting of a large, sweeping 'M' followed by a series of vertical strokes.

Muhammad Iqbal Raihan

A handwritten signature in black ink, featuring a large, bold 'M' followed by a series of vertical strokes.

Muhammad Akmal